

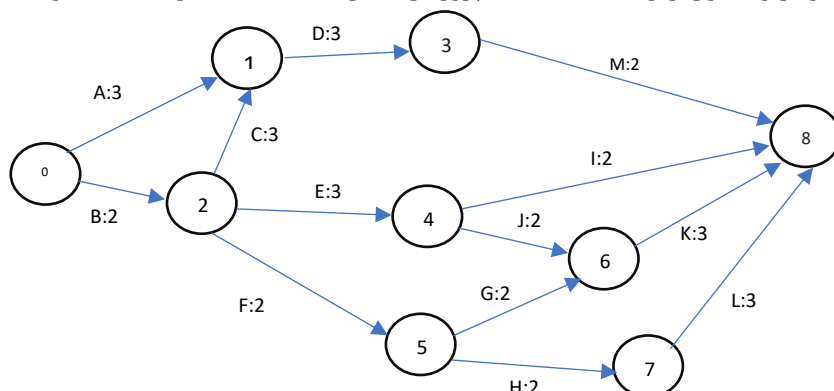


#	سوالات پشت و رو هست.	بارم
۱	در لایه روش‌ها ۴ مورد از آزمون‌ها را نام برده و با مثال کوتاه توضیح دهید.	۲
۲	چهار مورد از روش‌های توسعه نرم افزار را نام برده و توضیح دهید	۲
۳	چهار مورد از مسائل سازمانی انسانی و فنی را در توسعه نرم افزار چابک بیان و شرح دهید.	۲
۴	روش اسکرام از روش توسعه نرم افزار چابک را توضیح دهید و نمودار فرآیند اسکرام را بکشید.	۲
۵	اصل جانشینی لیسکوف از اصول برنامه نویسی SOLID را توضیح دهید و کد زیر را منطبق با این اصل نمائید. <pre>class Shape: def area(self): pass class Square(Shape): def __init__(self, side_length): self.side_length = side_length def area(self): return self.side_length * self.side_length class Circle(Shape): def __init__(self, radius): self.radius = radius def area(self): return 3.14 * self.radius * self.radius def calculate_total_area(shapes): total_area = 0 for shape in shapes: total_area += shape.area() return total_area shapes = [Square(5), Circle(3)] print("Total area:", calculate_total_area(shapes))</pre>	۳
	ادامه در پشت برگه	

۳

۶

نمودار و جدول زیر مشخصات یک پروژه را نشان می‌دهد. لطفا در ادامه به سوالات پاسخ مناسب دهید.



الف) مسیر بحرانی را مشخص کنید.

ب) زودترین زمان شروع T_E و دیرترین زمان شروع T_L را برای گره ۴ و گره ۱ و گره ۵ و گره ۶ محاسبه کنید.

ج) - اگر بخواهیم پروژه را یک هفته زودتر تمام کنیم، زودتر تمام کردن کدام فرآیند ها از نظر بودجه به بیشترین سربار هزینه را دارد و چقدر؟

فعالیت	هزینه	رابطه با زمان
A	۸۵	خطی معکوس
B	۱۸۰	خطی معکوس
C	۵۰	خطی مربع
E	۲۰۰	خطی معکوس
F	۱۱۰	خطی معکوس
D	۵۰	خطی مربع
I	۳۵	خطی معکوس
J	۴۰	خطی معکوس
G	۸۰	خطی مربع
H	۸۰	خطی معکوس
L	۷۰	خطی معکوس
K	۹۰	خطی مربع
M	۷۵	خطی مربع

موفق باشید-تقی نژاد

جواب ۱:

- **Unit Testing: آزمون واحد** (۱): تک تک اجزا (واحد‌ها) را به صورت مجزا بررسی کنید. (۲) مثال: الگوریتم جستجوی کتاب را برای نتایج دقیق آزمایش کنید.
- **Integration Testing: آزمون یکپارچگی** (۱): **اعتبار سنجی تعاملات** بین اجزای مختلف (۲) مثال: اطمینان حاصل کنید که افزودن کتاب به سبد خرید پایگاه داده را به درستی به روز می کند.
- **Acceptance Testing: آزمون پذیرش اعتبارسنجی سیستم در برابر الزامات تجاری**
- **Performance Testing: آزمون کارایی** (۱): پاسخگویی، مقیاس پذیری و استفاده از منابع را ارزیابی کنید. (۲) مثال: زمان پاسخگویی را در اوج ترافیک اندازه گیری کنید.
- **Security Testing: آزمون امنیت** (۱): آسیب پذیری ها را شناسایی کنید و از داده ها محافظت کنید. (۲) مثال: اطمینان از انتقال ایمن اطلاعات حساس.
- **Usability Testing: آزمون استفاده پذیری** (۱): کاربرپسندی را ارزیابی کنید. (۲) مثال: سهولت پیمایش و جریان خرید را ارزیابی کنید.

جواب ۲: **آبشاری**: این یک رویکرد خطی و متوالی است که در آن هر مرحله از فرآیند توسعه به ترتیب خاصی انجام می شود.

۱. **چابک**: این یک رویکرد تکراری و تطبیقی است که پروژه به واحدهای کوچک کار یا «داستان های کاربر» تقسیم می شود.

۲. **اسکرام**: این زیرمجموعه ای از چابک است که کار را به صورت اسپرینت های با زمان محدود سازماندهی می کند

• **توسعه سریع برنامه RAD**: این یک رویکرد سریع و انعطاف پذیر است که بر نمونه سازی سریع و تحویل تکراری تأکید دارد.

□ تیم وارد فازهای نمونه سازی سریع می شود.

جواب ۳: ۱) **انتخاب رویکرد مناسب**: فرآیندهای مبتنی بر برنامه و چابک

- آیا داشتن **مشخصات و طراحی** بسیار دقیق قبل از رفتن به پیاده سازی مهم است؟ در این صورت، احتمالاً باید از **رویکرد مبتنی بر برنامه** استفاده کنید.
- آیا **استراتژی تحویل افزایشی** که نرم افزار را به مشتریان تحویل می دهید و **بازخورد سریع** از آنها دریافت می کنید، واقع بینانه است؟ در این صورت، استفاده از **روش های چابک** را در نظر بگیرید.
- **اندازه سیستمی** که در حال توسعه است چقدر است؟ **روش های چابک** زمانی مؤثرتر هستند که سیستم بتواند با یک **تیم کوچک** هم مکان که می توانند به صورت غیررسمی ارتباط برقرار کنند، توسعه یابد.
- سیستم بزرگ -> **ریکرد مبتنی بر برنامه**
- ۲) **نوع سیستمی که در حال توسعه است**: رویکردهای مبتنی بر برنامه ممکن است برای سیستم هایی که قبل از پیاده سازی نیاز به تجزیه و تحلیل زیادی دارند (به عنوان مثال، سیستم بلادرنگ با الزامات زمان بندی پیچیده) مورد نیاز باشد.
- ۳) **عمر مفید مورد انتظار سیستم**: سیستم های با **عمر طولانی** ممکن است به **مستندات طراحی** بیشتری نیاز داشته باشند تا بتوانند اهداف اصلی توسعه دهندگان سیستم را به تیم پشتیبانی منتقل کنند.
- ۴) **فناوری های موجود برای پشتیبانی از توسعه سیستم**: روش های چابک برای پیگیری یک طراحی در حال تکامل به ابزارهای خوبی نیاز دارند.
- ۵) **نحوه سازماندهی تیم توسعه**: اگر تیم توسعه توزیع شده باشد یا بخشی از توسعه برون سپاری شده باشد، ممکن است نیاز به تهیه **اسناد طراحی** برای برقراری ارتباط بین تیم های توسعه داشته باشید.

جواب ۴: رویکرد اسکرام یک روش چابک کلی است اما تمرکز آن بر مدیریت توسعه تکرارشونده **Iterative Development** به جای شیوه های چابک خاص است.

• اسکرام دارای سه فاز است.

• فاز اولیه، فاز برنامه ریزی کلی است که در آن اهداف کلی پروژه را تعیین کرده و معماری نرم افزار را طراحی می کنید.



- این مرحله به دنبال مجموعه‌ای از چرخه‌های اسپرینت **Sprint** است که در هر چرخه، افزایشی از سیستم توسعه می‌یابد.
- فاز **اختتامیه** پروژه، پروژه را جمع بندی می کند، مستندات مورد نیاز مانند چارچوب های راهنمای سیستم و دستورالعمل های کاربر را تکمیل می کند و به ارزیابی درس های آموخته شده از پروژه می پردازد.



۵- اصل جانشینی لیسکوف (LSP) این اصل بیان می کند که اشیاء یک زیرکلاس باید بدون اینکه بر صحت برنامه تأثیر بگذارد، قابل جایگزینی با اشیاء سوپرکلاس آن باشند.

- مثال: فرض کنید سناریویی دارید که در آن یک سلسله مراتب کلاسی برای اشکال دارید، از جمله یک کلاس پایه به نام «شکل» **Shape** و زیرکلاس هایی مانند «مربع» و «دایره»

```
class Shape:
    def area(self):
        pass

class Square(Shape):
    def __init__(self, side_length):
        self.side_length = side_length

    def area(self):
        return self.side_length * self.side_length

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius

def calculate_total_area(shapes):
    total_area = 0
    for shape in shapes:
        total_area += shape.area()
    return total_area

shapes = [Square(5), Circle(3)]
print("Total area:", calculate_total_area(shapes))
```

جواب ۶

الف (مسیر B-E-J-K

ب: گره ۴: 5,5 – گره ۱: 3,5 و گره ۳: 4,5 و گره ۶: 7,7

ج: C یا د